

```

0000 separate (IDL.SEM_PHASE) use VIS_UTIL; -- FOR DEBUG (NODE_REP) use DEF_UTIL; use NOD_WALK;
procedure REDECL_SOURCE_NAME (SOURCE_NAME : in out TREE; NODE_HASH : in out NODE_HASH_TYPE; H_IN :
: H_TYPE; DECL : TREE := TREE_VOID) is
-- MAKES A NEW SOURCE NAME FOR A DECLARATION CREATED BY INSTANTIATION -- ... MUST
BE CAREFUL TO SUBSTITUTE FOR CONSTITUENTS IN THE -- ... PROPER-ORDER SO THAT DECLARATIONS ARE PROCESSED BEFORE USED -- ... (EVENTUALLY,
THIS PROCEDURE COULD BE EXTENDED TO SUBSTITUTES -- ... FOR GENERIC BODIES) OLD_NAME := SOURCE_NAME; DEF := TREE; TREE := T
REE_VOID; H := H_TYPE; H_IN := H_IN; S := S_TYPE; begin
... IN THE CURRENT COMPILATION if D(LX_SYMPREP, SOURCE_NAME).TY = DN_TXTREP then D(LX_SYMPREP, SOURCE_NAME, STORE_SYM( PRINT_NAME( D(LX_SY
MREP, SOURCE_NAME) ) ) ); end if; GEN_SUBS.REPLACE_NODE( SOURCE_NAME, NODE_HASH ); case CLASS_SOURCE_NAME( SOURCE_NAME.TY ) is
-- FOR A VARIABLE ID when DN_VARIABLE_ID => declare TYPE_SPEC := TREE; begin MAKE A-DEF FOR IT DEF := MAKE_DEF_FOR_ID(
SOURCE_NAME, H ); MAKE_DEF_VISIBLE( DEF ); IF THIS VARIABLE IS DECLARED BY A TASK DECLARATION if DECL.TY = DN_TASK_DECL then
0010 IN THE TASK REGION, MAKE NEW-TYPE AND SUBSTITUTE ENTER_REGION( DEF, H, S ); TYPE_SPEC := D(SM_OBJ_TYPE, SOURCE_NAME ); REPLACE_NODE( TYPE_S
PEC, NODE_HASH ); SUBSTITUTE_ATTRIBUTES( TYPE_SPEC, NODE_HASH, H ); LEAVE_REGION( DEF, S ); H := H_IN; -- ELSE IF DECLARATION IN
CLUDES-A CONSTRAINED ARRAY-DEFINITION elsif DECL.TY = DN_RENAMES_OBJ_DECL and then D(AS_TYPE_DEF, DECL).TY = DN_CONSTRAINED_ARRAY_DEF then
-- MAKE A NEW BASE TYPE TYPE_SPEC := GET_BASE_TYPE( D(SM_OBJ_TYPE, SOURCE_NAME) ); REPLACE_NODE( TYPE_SPEC, NODE_HASH ); SUBSTITUTE_A
TTIBUTES( TYPE_SPEC, NODE_HASH, H ); end if; -- FOR A CONSTANT ID when DN_CONSTANT_ID => declare FIRST_NAME := TRE
E := SOURCE_NAME; TYPE_SPEC := TREE; INIT_EXP := TREE; begin IF THIS IS THE DEFINING OCCURRENCE FIRST_NAME := D(SM_FIRST, OLD_NAM
E ); if FIRST_NAME = OLD_NAME then FIRST_NAME := SOURCE_NAME; -- MAKE A DEF FOR IT DEF := MAKE_DEF_FOR_ID( SOURCE_NAME, H ); MA
KE_DEF_VISIBLE( DEF ); -- IF THIS IS A DEFERRED CONSTANT DECLARATION if DECL.TY = DN_DEFERRED_CONSTANT_DECL then -- CLEAR
THE INITIAL EXPRESSION D(SM_INIT_EXP, SOURCE_NAME, TREE_VOID) ; -- ELSE IF DECLARATION INCLUDES A-CONSTRAINED ARRAY DEF elsif D(
AS_TYPE_DEF, DECL).TY = DN_CONSTRAINED_ARRAY_DEF then -- MAKE A NEW-BASE TYPE TYPE_SPEC := GET_BASE_TYPE( D(SM_TYPE_SPEC, SOURCE_NA
ME) ); REPLACE_NODE( TYPE_SPEC, NODE_HASH ); SUBSTITUTE_ATTRIBUTES( TYPE_SPEC, NODE_HASH, H ); end if; -- ELSE -- SINCE TH
IS-IS NOT THE DEFINING OCCURRENCE else SUBSTITUTE( FIRST_NAME, NODE_HASH, H ); -- FIX UP FORWARD REFERENCE IN THE DEFERRED CONSTANT
INIT_EXP := D(SM_INIT_EXP, SOURCE_NAME ); SUBSTITUTE( INIT_EXP, NODE_HASH, H ); D(SM_INIT_EXP, FIRST_NAME, INIT_EXP ); end if; end;
0020 -- FOR A DISCRIMINANT ID when DN_DISCRIMINANT_ID => declare FIRST_NAME := D(SM_FIRST, OLD_NAME ); begin IF THIS IS THE DEFINING OCCURRENCE if FIRST_NAME = OLD_NAME then -- MAKE A DEF FOR IT DEF := MAKE_DEF_FOR_ID( SOURCE_NAME, H ); M
AKE_DEF_VISIBLE( DEF ); end if; -- FOR AN ENUMERATION LITERAL when CLASS_ENUM_LITERAL => declare HEADER := TREE :=
TREE_VOID; DEFLIST := SEQ_TYPE := LIST( D(LX_SYMPREP, SOURCE_NAME) ); DEF := TREE; begin IF NAME IS USED while not IS_EMPTY( DEFLIS
T ) loop POP( DEFLIST, DEF ); if D(XD_SOURCE_NAME, DEF) = OLD_NAME then HEADER := D(XD_HEADER, DEF ); exit; end if; end loop;
if HEADER = TREE_VOID then -- GET AND SUBSTITUTE IN THE OLD HEADER HEADER := D(XD_HEADER, GET_DEF_FOR_ID( OLD_NAME )); SUBSTITUTE
E( HEADER, NODE_HASH, H ); -- MAKE A DEF FOR THE NEW SOURCE NAME DEF := MAKE_DEF_FOR_ID( SOURCE_NAME, H ); MAKE_DEF_VISIBLE( DEF, H
EADER ); end if; -- FOR A TYPE ID when DN_TYPE_ID => declare FIRST_NAME := SOURCE_NAME; TYPE_SPEC := TREE;
0030 begin GET THE ORIGINAL TYPE_SPEC AND DEFINING OCCURRENCE TYPE_SPEC := D(SM_TYPE_SPEC, OLD_NAME ); FIRST_NAME := D(SM_FIRST, OLD_
NAME ); IF THIS IS THE DEFINING OCCURRENCE if FIRST_NAME = OLD_NAME then FIRST_NAME := SOURCE_NAME; -- MAKE A DEF FOR IT DEF :=
MAKE_DEF_FOR_ID( SOURCE_NAME, H ); MAKE_DEF_VISIBLE( DEF ); -- CLEAR ANY FORWARD REFERENCE TO FULL TYPE SPEC if TYPE_SPEC.TY
IN CLASS_CONSTRAINED then TYPE_SPEC := D(SM_BASE_TYPE, TYPE_SPEC ); end if; if TYPE_SPEC.TY = DN_INCOMPLETE then D(XD_FULL_TYPE_SPEC
, TYPE_SPEC, TREE_VOID ); else TYPE_SPEC := GET_BASE_TYPE( TYPE_SPEC ); end if; end if; -- ELSE -- SINCE THIS IS NOT THE DEFINING OCCU
RRENCE else SUBSTITUTE( FIRST_NAME, NODE_HASH, H ); -- GET THE EXISTING DEF DEF := GET_DEF_FOR_ID( FIRST_NAME ); end if;
GET AND REPLACE THE TYPE_SPEC NODE FOR THE BASE TYPE TYPE_SPEC := GET_BASE_TYPE( TYPE_SPEC ); REPLACE_NODE( TYPE_SPEC, NODE_HASH );
IF THIS TYPE IS (POSSIBLY)-A DECLARATIVE REGION if TYPE_SPEC.TY = DN_RECORD or TYPE_SPEC.TY = DN_TASK_SPEC then -- WBE 7/31/90
0040 ENTER_REGION AND SUBSTITUTE WITHIN THE TYPE SPEC ENTER_REGION( DEF, H, S ); -- ELSE -- SINCE THIS-TYPE CANNOT BE A DECLARATIVE REGION else
ERATION TYPE if TYPE_SPEC.TY = DN_ENUMERATION then -- MAKE NEW ENUMERATION-LITERALS declare LITERAL_LIST := LIST
( D(SM_LITERAL_S, TYPE_SPEC) ); LITERAL := TREE; begin while not IS_EMPTY( LITERAL_LIST ) loop POP( LITERAL_LIST, LITERAL
); REPLACE_SOURCE_NAME( LITERAL, NODE_HASH, H ); end loop; end if; end if; -- SUBSTITUTE WITHIN THE TYPE SPEC SU
BSTITUTE_ATTRIBUTES( TYPE_SPEC, NODE_HASH, H ); end if; IF THIS WAS NOT A DEFINING OCCURRENCE if FIRST_NAME /= SOURCE_NAME then
-- GET AND SUBSTITUTE IN THE FULL SUBTYPE TYPE_SPEC := D(SM_TYPE_SPEC, SOURCE_NAME ); SUBSTITUTE( TYPE_SPEC, NODE_HASH, H );
-- F
IX UP FORWARD REFERENCES IN-DEFINING OCCURRENCE if FIRST_NAME.TY = DN_TYPE_ID then D(XD_FULL_TYPE_SPEC, D(SM_TYPE_SPEC, FIRST_NAME ), TYPE_S
PEC ); else D(SM_TYPE_SPEC, D(SM_TYPE_SPEC, FIRST_NAME ), TYPE_SPEC ); end if; end if; -- FOR AN [L]PRIVATE TYPE ID
when DN_PRIVATE_TYPE_ID | DN_PRIVATE_TYPE_ID => declare TYPE_SPEC := TREE; begin MAKE A-DEF FOR IT DEF := MAKE_DEF_F
OR_ID( SOURCE_NAME, H ); MAKE_DEF_VISIBLE( DEF ); -- REPLACE THE TYPE_SPEC NODE FOR THE BASE TYPE TYPE_SPEC := D(SM_TYPE_SPEC, SOURCE_NAME );
0050 -REPLACE_NODE( TYPE_SPEC, NODE_HASH ); -- CLEAR FORWARD REFERENCE TO-FULL TYPE-SPEC D(SM_TYPE_SPEC, TYPE_SPEC, TREE_VOID ); -- ENTER REGIO
N AND-SUBSTITUTE WITHIN THE TYPE SPEC-ENTER_REGION( DEF, H, S ); SUBSTITUTE_ATTRIBUTES( TYPE_SPEC, NODE_HASH, H ); LEAVE_REGION( DEF, S ); H := RE
PLACE_SOURCE_NAME.H_IN; end; -- FOR A UNIT OR ENTRY NAME when CLASS_NON_TASK_NAME | DN_ENTRY_ID => declare HEADER := TREE
:= D(SM_SPEC, SOURCE_NAME ); UNIT_DESC := TREE_VOID; DECL_S := TREE; NOT_EQUAL := TREE; begin GET THE UNIT_DESC FROM THE-DEC
LARATION ... (DECL VOID FOR "/" OR-DERIVED FUNCTION) if DECL := TREE_VOID and then SOURCE_NAME.TY IN CLASS_SUBPROG_PACK_NAME then UNIT_DES
C := D(SM_UNIT_DESC, SOURCE_NAME ); end if; MAKE DEF AND ENTER REGION-DEF := MAKE_DEF_FOR_ID( SOURCE_NAME, H ); ENTER_REGION( DEF, H, S );
H.IS_IN_SPEC := False; IF THIS IS AN INSTANTIATION if UNIT_DESC.TY = DN_INSTANTIATION then -- SUBSTITUTE FOR THE-DECLARATIONS
OF-THE-GENERIC ACTUALS DECL_S := D(SM_DECL_S, UNIT_DESC ); SUBSTITUTE( DECL_S, NODE_HASH, H ); -- ELSE IF THIS IS A GENERIC DECLAR
ATION elsif SOURCE_NAME.TY = DN_GENERIC_ID then -- CLEAR THE FORWARD REFERENCE D(SM_BODY, SOURCE_NAME, TREE_VOID );
0060 SUBSTITUTE FOR THE-HEADER if HEADER.TY = DN_PACKAGE_SPEC then DECL_S := D(AS_DECL_S, HEADER ); H.IS_IN_SPEC := True; SUBSTITUTE( DECL
S, NODE_HASH, H ); H.IS_IN_SPEC := False; elsif HEADER.TY = DN_TASK_SPEC then H.IS_IN_SPEC := True; end if; SUBSTITUTE( HEADER, NODE_HASH, H );
-- MAKE THE DEF VISIBLE if SOURCE_NAME.TY IN CLASS_SUBPROG_NAME then MAKE_DEF_VISIBLE( DEF, HEADER ); else MAKE_DEF_VISIBLE( DEF, H )
nd if; -- LEAVE REGION-LEAVE_REGION( DEF, S ); H := REPLACE_SOURCE_NAME.H_IN; -- IF THIS IS AN OPERATOR ID FOR "=" if SOURCE_NAME.TY = DN
_OPERATOR_ID and then D(XD_NOT_EQUAL, SOURCE_NAME) /= TREE_VOID then -- REPLACE THE INEQUALITY OPERATOR TO00 NOT_EQUAL := D(XD_NOT_E
QUAL, SOURCE_NAME ); REPLACE_SOURCE_NAME( NOT_EQUAL, NODE_HASH, H, TREE_VOID ); D(XD_NOT_EQUAL, SOURCE_NAME, NOT_EQUAL ); end if;
end; -- FOR ID'S WITH NO SPECIAL STRUCTURE when DN_NUMBER_ID | DN_COMPONENT_ID | CLASS_PARAM_NAME | DN_SUBTYPE_ID | DN_EXCEPTION_ID =>
-- MAKE-A DEF IF NAME IS USED if D(LX_SYMPREP, SOURCE_NAME).TY = DN_SYMBOL_REP then DEF := MAKE_DEF_FOR_ID( SOURCE_NAME, H ); MAKE_DEF_VISI
BLE( DEF ); end if; -- FOR ID'S WHICH SHOULD NOT OCCUR when DN_ITERATION_ID | DN_TASK_BODY_ID | CLASS_LABEL_NAME => abort
THE COMPILATION PUT_LINE( "NEWSNAM.REPLACE_SOURCE_NAME : INVALID ID FOR GENERIC SUBSTITUTION" ); raise Program_Error; end case;
0070 -- SUBSTITUTE-FOR THE ATTRIBUTES OF THE ID SUBSTITUTE_ATTRIBUTES( SOURCE_NAME, NODE_HASH, H ); end --=====
===== end-NEWSNAM; end

```